Angular Lifecycle Hooks

Explained – Real Use Cases, Pros & Pitfalls

/sreeramsaikumar

Sreeram Saikumar
Angular Developer

What is Component Lifecycle?

The Component Lifecycle in Angular refers to the sequence of events from the creation to the destruction of a component.

Angular components go through a lifecycle:

Initialization → Change detection → DOM rendering → Destruction

```
Create → Render → Update → Destroy
```

Complete Lifecycle Hook Order

1. ngOnChanges()
2. ngOnInit()
3. ngDoCheck()
4. ngAfterContentInit()
5. ngAfterContentChecked()
6. ngAfterViewInit()
7. ngAfterViewChecked()
8. ngOnDestroy()

1. ngOnChanges()

ngOnChanges() is called whenever an **@Input()** property changes — and before **ngOnInit()** runs. It receives a **SimpleChanges** object with the previous and current values of each input.

When to use:

- When your component depends on dynamic input data from the parent.
- Useful for triggering logic based on input changes, such as updating form fields or calling services.

When not to:

- If the input won't change after the component initializes — better use **ngOnInit()**.
- Don't use it for change detection of deep objects (like nested arrays/objects) — use **ngDoCheck()** instead.

Pros:

- Provides both previousValue and currentValue — great for tracking deltas.
- Useful in reusable components that rely heavily on dynamic inputs.

Cons / Caveats:

- Doesn't fire on first component creation if input value doesn't change.
- It only checks for reference changes, not deep equality (=== comparison).

```
@Input() user!: User;

ngOnChanges(changes: SimpleChanges) {
  if (changes['user']) {
    const newUser = changes['user'].currentValue;
    this.initUserForm(newUser);
  }
}
```

(Best for reacting to parent-driven input changes without needing to watch deeply nested objects.)

2. ngOnInit

Called once after the first **ngOnChanges()** and after Angular sets all input properties. Think of it as your component's initialization point.

When to use:

- Fetch data from services or APIs.
- Set up default values, listeners, or local state.
- Logic that depends on **@Input()** but only needs to run once.

When not to:

- Don't access the DOM here — the view is not initialized yet.
- Avoid logic that needs to run on every input change — use **ngOnChanges()** for that.

Pros:

- Safe place to initialize without worrying about change detection.
- Input properties are set and ready.

Cons / Caveats:

- Only called once — not suitable for responding to future changes.
- No access to **@ViewChild** DOM elements yet.

```
ngOnInit() {
  this.fetchUser();
  this.theme = this.settingsService.getTheme();
}
```

(Use this as your reliable “componentDidMount” to bootstrap business logic.)

3. ngDoCheck()

Called every time Angular runs change detection, allowing you to perform custom change tracking.

When to use:

- You need to track changes in mutable or nested objects/arrays that Angular doesn't detect (reference doesn't change).
- Perform manual dirty checking or complex comparison.

When not to:

- Avoid unless necessary — it runs very frequently, so be performance-conscious.
- Don't replace **ngOnChanges()** with this if input change detection is enough.

Pros:

- Full control over change tracking — great for deeply nested data.
- Can complement or override Angular's default detection.

Cons / Caveats:

- Can cause performance issues if misused.
- Angular won't optimize this — it runs on every CD cycle.

```
ngDoCheck() {
  if (this.oldList.length !== this.newList.length) {
    this.handleChange();
  }
}
```

(Only use when default detection is insufficient. Combine with **ChangeDetectorRef** if needed.)

4. ngAfterContentInit()

Called once after Angular projects external content (via **<ng-content>**) into the component's view.

When to use:

- Access and manipulate projected content using @ContentChild.
- Useful for content-wrapping components like tabs, modals, or custom wrappers.

When not to:

- Don't use this for DOM elements declared in your own template — use ngAfterViewInit() for that.
- Avoid logic that depends on repeated content changes — this only runs once.

Pros:

- Ensures projected content is ready and available for use.
- Great for content projection-based APIs.

Cons / Caveats:

- Doesn’t re-run if content changes dynamically.
- Confused often with **ngAfterViewInit()** — use it only for external/projection content.

```
@ContentChild('header') header!: ElementRef;

ngAfterContentInit() {
  console.log(this.header.nativeElement.innerText);
}
```

(If you use **<ng-content>**, this is the hook you’ll rely on.)

5. ngAfterContentChecked()

Called after every change detection cycle on projected content. Runs more often than **ngAfterContentInit()**.

When to use:

- You want to validate or update projected content regularly.
- Used rarely, mostly in low-level libraries or deeply dynamic UIs.

When not to:

- Avoid unless your component depends on frequent content updates.
- Avoid expensive operations — this runs frequently.

Pros:

- Keeps your content state in sync.
- Can detect external content mutations.

Cons / Caveats:

- Difficult to optimize — no diffing system.
- Rarely used unless creating framework-like libraries.


```
ngAfterContentChecked() {
  if (this.projectedText !== this.previousText) {
    this.previousText = this.projectedText;
    this.reprocessContent();
  }
}
```

(Rarely used. But essential when working with highly dynamic **<ng-content>**.)

6. ngAfterViewInit()

Called once after the component’s view (template) and its child components are fully initialized.

When to use:

- Access **@ViewChild()** or @**ViewChildren()** elements.
- Perform DOM manipulations or call component methods.

When not to:

- Don’t perform data fetches here — prefer **ngOnInit()**.
- Avoid triggering change detection from here — can lead to **ExpressionChangedAfterItHasBeenCheckedError**.

Pros:

- View is fully initialized — safe to access the DOM.
- You can safely interact with components inside your template.

Cons / Caveats:

- Runs only once — won't detect future view changes.
- Avoid triggering side effects that modify the view.

```
@ViewChild('searchInput') input!: ElementRef;

ngAfterViewInit() {
  this.input.nativeElement.focus();
}
```

(Best hook for DOM and template interactions using **@ViewChild**.)

7. ngAfterViewChecked()

Called after Angular checks the component's view and child views — and after any change detection runs.

When to use:

- Track or validate DOM changes after detection.
- Rarely used — mostly for edge cases or third-party integration.

When not to:

- Avoid changing the DOM here — can cause infinite loops.
- Don't trigger CD manually from this hook.

Pros:

- Allows post-render checks for layout, visibility, etc.

Cons / Caveats:

- Runs after every CD cycle — can degrade performance.
- Risk of expression-changed errors.

```
ngAfterViewChecked() {
  if (this.lastHeight !== this.div.nativeElement.offsetHeight) {
    this.resizeHandler();
  }
}
```

(Advanced use only. Be careful of re-render loops.)

8. ngOnDestroy()

Called right before Angular destroys the component. Use it to **clean up** resources.

When to use:

- Unsubscribe from Observables or subjects.
- Clear timers, event listeners, and socket connections.

When not to:

- Don’t place component logic here.
- Never skip it if you’re subscribing manually.

Pros:

- Prevents memory leaks and zombie subscriptions.
- Essential for long-living applications or routing-based views.

Cons / Caveats:

- Easy to forget if you don’t track subscriptions properly.
- Angular doesn't force you to implement it — you must be disciplined.

```
ngOnDestroy() {
  this.subscription.unsubscribe();
  clearInterval(this.pollingInterval);
}
```

(Always use this for resource cleanup in every component or service.)

/sreeramsaikumar

+ Follow

/sreeramsaikumar